UNIVERSIDAD TECNOLOGICA DE TEHUACAN

PROGRAMA EDUCATIVO EN:

Tecnologías de la Información en Área Desarrollo
de Software Multiplataforma

NOMBRE DEL PROFESOR:

JOSÉ MIGUEL CARRERA PACHECO

INTEGRANTES DEL EQUIPO:

Ciro Julian Cervantes Zamora

MATERIA:

Desarrollo Web Profesional

TEMA:

Manejo de multisesiones y persistencia

TEHUACÁN, PUEBLA, ENERO-ABRIL 2026

1. ¿Qué es una multisesión?

Es la capacidad del sistema de permitir que un mismo usuario tenga múltiples sesiones activas simultáneamente en diferentes pestañas, navegadores o dispositivos.

Ejemplo de uso:

Un usuario inicia sesión en su laptop, luego abre otra pestaña, y también accede desde su celular. Todas comparten el mismo usuario pero son sesiones independientes.

2. Problemas Resueltos

La siguiente tabla describe los principales problemas detectados y las soluciones implementadas en el sistema:

Problema	Solución implementada
Sesiones infinitas acumuladas	Límite de 5 sesiones por usuario. Al superar el límite, se elimina la más antigua.
Sesiones expiradas activas	El sistema limpia sesiones expiradas antes de cada validación (cleanExpiredSessions).
Pérdida de conexión	Al reconectar (online event), se verifica si la sesión sigue válida con /auth/verify.
Inconsistencia entre pestañas	Se usa el evento storage de localStorage para sincronizar estado entre pestañas.

3. Flujo de Multisesión

El siguiente flujo describe cómo se gestionan las sesiones en el sistema:

- Usuario abre pestaña 1 → Login → Token guardado en localStorage → Sesión guardada en backend (array)
- Usuario abre pestaña 2 → Mismo login → Nuevo token → Sesión #2 en backend
- Usuario abre pestaña 3 → Nuevo login → Sesión #3 en backend
- ...
- Sesión #6 → Se elimina la sesión más antigua → Se crea la nueva

4. Invalidación de Sesión

4.1 Logout manual

El usuario cierra sesión desde cualquier pestaña:

- Se elimina la sesión del array en backend
- Se borra el token de localStorage
- El evento storage sincroniza todas las demás pestañas

4.2 Logout forzado (otra pestaña)

- Al borrar el token en una pestaña, las demás detectan el cambio vía storage event
- Se muestra modal “SESIÓN TERMINADA”

4.3 Expiración automática

- El JWT expira en 8 horas
- Si el JWT es inválido o no existe en el array de sesiones, se rechaza la petición con error 401

5. Riesgos sin Control de Multisesión

Los siguientes riesgos pueden presentarse si no se implementa un control adecuado de multisesión:

Riesgo	Consecuencia
Sesiones sin límite	Memoria agotada, degradación de rendimiento
No validar sesión en cada request	Usuarios con acceso no autorizado
No sincronizar pestañas	Confusión: usuario ve “sesión activa” en una pestaña, “sesión cerrada” en otra
Sin expiración	Sesiones “zombis” que nunca se liberan
Sin logout global	Usuario cierra sesión en un dispositivo pero sigue activo en otros

6. Endpoints Relacionados

Método	Endpoint
POST	/api/auth/login
POST	/api/auth/logout
GET	/api/auth/verify

7. Estructura de una Sesión (Backend)

Cada sesión activa se almacena en el backend con la siguiente estructura JSON:

```
{
  userId: 1,
  token: "jwt...",
  expiresAt: Date.now() + 8 * 60 * 60 * 1000
}
```

El campo expiresAt corresponde a 8 horas desde el momento del login.

8. Diagrama de Flujo

El siguiente diagrama ilustra el proceso de creación y validación de sesiones:

LOGIN

↓

¿Límite de 5 sesiones alcanzado?

↓ Sí: eliminar sesión más antigua ↓ No: continuar

Crear nueva sesión

↓

Middleware valida en cada request

↓

Válida → Continuar | Expirada → Error 401 | Invalidada → Error 401